
JAL240I
PRINCIPLES OF ARTIFICIAL INTELLIGENCE

UNIT II
PROBLEM SOLVING AND SEARCHING

UNIT II
PROBLEM SOLVING AND SEARCHING

SYLLABUS

Solving problems by searching - Problem-Solving agents - Example problems - Search algorithms - Uninformed search strategies - Informed (Heuristic) search strategies - Heuristic functions - Search in complex environments - Local search and optimization problems - Local search in continuous spaces - Search with nondeterministic actions - Search in partially observable environments - Online search agents and unknown environments.

2

INTRODUCTION

- **Problem-solving agent** → An agent that may need to plan ahead when the correct action to take is not immediately obvious. Such an agent has to consider a sequence of actions that form a path to a goal state.
- **Search** → the computational process a problem-solving agent undertakes is called search.
- **Informed search algorithms** → algorithms in which the agent can estimate how far it is from the goal → eg. Greedy best-first search, A* search, search contours, Satisficing search, Memory-bounded search, Bidirectional heuristic search.
- **Uninformed algorithms** → algorithms where no estimate on the distance to goal is available → eg. BFS, DFS, , Dijkstra's algorithm or uniform-cost search, Depth-limited and iterative deepening search, Bidirectional search.

PROBLEM SOLVING AGENTS

In a known environment, the agent can follow the **four-phase problem-solving process**:

- **Goal formulation:** The agent adopts the goal of reaching a particular destination. Goals organize behavior by limiting the objectives and hence the actions to be considered.
- **Problem formulation:** The agent devises a description of the states and actions necessary to reach the goal—an abstract model of the relevant part of the world.
- **Search:** Before taking any action in the real world, the agent simulates sequences of actions in its model, searching until it finds a sequence of actions that reaches the goal. Such a sequence is called a solution. Multiple sequences simulated, but eventually a solution will be found, or it will find that no solution is possible.
- **Execution:** The agent can now execute the actions in the solution, one at a time.

PROBLEM SOLVING AGENTS

- In a fully observable, deterministic, known environment, the solution to any problem is a **fixed sequence** of actions.
- If the model is correct, then once the agent has found a solution, it can ignore its percepts while it is executing the actions → **open-loop system** → ignoring the percepts breaks the loop between agent and environment.
- If there is a chance that the model is incorrect, or the environment is nondeterministic, then the agent would be safer using a **closed-loop approach** that monitors the percepts.
- In partially observable or nondeterministic environments, a solution would be a **branching strategy** that recommends different future actions depending on what percepts arrive.

SEARCH PROBLEMS AND SOLUTIONS

A search problem → defined to have the following:

- A set of possible **states** that the environment can be in → also called as state space.
- The **initial state** → the state that the agent starts in.
- **Goal states** → the state (s) to be achieved → one or more.
- **Actions** available to the agent → Given a state s , $ACTIONS(s)$ returns a finite actions that can be executed in s → applicable in state s .
- A **transition model** → describes what each action does. $RESULT(s, a)$ returns the state that results from doing action a in state s .
- An **action cost function** - $ACTION-COST(s, a, s')$ or $c(s, a, s')$ → gives the numeric cost (EG. Time, distance, money spent) of applying action a in state s to reach state s' → used as a performance measure by an agent.

SEARCH PROBLEMS AND SOLUTIONS

- A **search algorithm** → takes a search problem as input and returns a solution, or an indication of failure.
- **Path** → a sequence of actions.
- **Solution** → a path from the initial state to a goal state.
- The **total cost of a path** = the sum of the individual action costs
- An **optimal solution** → the solution that has the lowest path cost among all solutions.
- The state space can be represented as a **graph** in which the vertices are states and the directed edges between them are actions.

SEARCH PROBLEMS AND SOLUTIONS

- **Eg. Route finding problem:**
- **States** → locations and current time
- **Initial state** → home (source) airport
- **Goal state** → destination city (airport)
- **Actions** → Take any flight from the current location, in any seat class, leaving after the current time, leaving enough time for within-airport transfer if needed.
- **Transition model** → The state resulting from taking a flight will have the flight's destination as the new location and the flight's arrival time as the new time.
- **Action cost** → A combination of monetary cost, waiting time, flight time, customs and immigration procedures, seat quality, time of day, type of airplane, frequent-flyer reward points, and so on.
- Contingency plans — for delayed flight and missed connection

FORMULATING PROBLEMS

- The process of removing detail from a representation is called **abstraction**. A good problem formulation has the right level of detail.
- **Eg. Route finding problem:** “move the right foot forward a centimeter” or “turn the steering wheel one degree left” → not so useful in routing to a particular destination

EXAMPLE PROBLEMS

1. A **standardized problem** is intended to illustrate or exercise various problem-solving methods. It can be given a concise, exact description and hence is suitable as a benchmark for researchers to compare the performance of algorithms. Eg. Chess game
2. A **real-world problem**, is one whose solutions people actually use, and whose formulation is idiosyncratic, not standardized, because, for example, each robot has different sensors that produce different data. Eg. robot navigation

EXAMPLE PROBLEMS

1. Standardized problems

A grid world problem → two-dimensional rectangular array of square cells in which agents can move from cell to cell.

- Actions → pick up, push, or otherwise act upon.

Vacuum world:

- States: A state of the world says which objects are in which cells → dirt or agent → with n cells $n2^n$ states
- Initial state: Any state can be designated as the initial state.
- Actions: three actions: Suck, move Left, and move Right.
- Transition model: Suck removes any dirt from the agent's cell; Forward moves the agent ahead one cell in the direction it is facing, unless it hits a wall, in which case the action has no effect. Backward moves the agent in the opposite direction, while TurnRight and TurnLeft change the direction it is facing by 90.
- Goal states: The states in which every cell is clean.
- Action cost: Each action costs 1.

EXAMPLE PROBLEMS

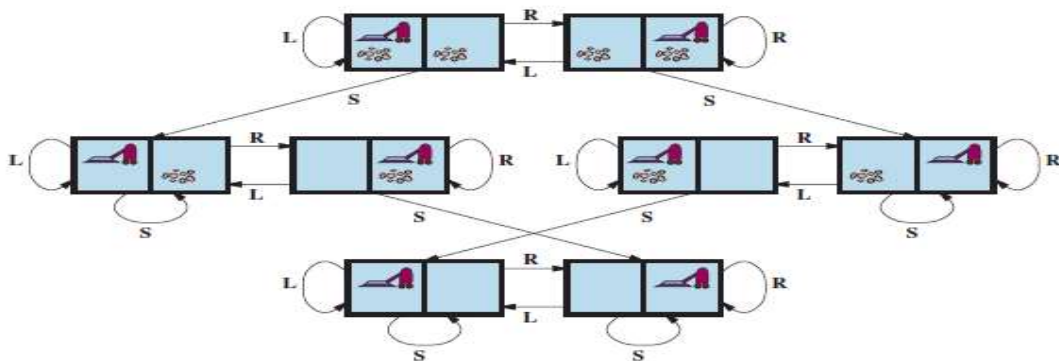


Figure 3.2 The state-space graph for the two-cell vacuum world. There are 8 states and three actions for each state: L = Left, R = Right, S = Suck.

EXAMPLE PROBLEMS

1. Standardized problems

Sokoban puzzle → agent's goal is to push a number of boxes, scattered about the grid, to designated storage locations.

- Actions → push a box or move forward to an empty cell.
- States: A state of the world says which objects are in which cells → empty or contains a box → For a world with n non-obstacle cells and b boxes, there are $n \times n! / (b!(n - b)!)$ states

EXAMPLE PROBLEMS

7	2	4
5		6
8	3	1

Start State

	1	2
5	4	3
6	7	8

Goal State

A typical instance of the 8-puzzle.

1. Standardized problems

Sliding Tile Puzzle → number of tiles (sometimes called blocks or pieces) are arranged in a grid with one or more blank spaces so that some of the tiles can slide into the blank space.

Eg. 8-puzzle or 15-puzzle

- States: specifies the location of each of the tiles
- Initial state: Any state can be designated as the initial state.
- Actions: blank space moving Left, Right, Up, or Down..
- Transition model: Maps a state and action to a resulting state. Applying Left to the start state, the resulting state has the 5 and the blank switched
- Goal states: Any state with the numbers in order could be the goal.
- Action cost: Each action costs 1.

EXAMPLE PROBLEMS

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

A typical instance of the 8 puzzle

2. Real-world problems

Route-finding Problem → specified locations and transitions along edges between them.

Applications → Web sites and in-car systems, routing video streams in computer networks, military operations planning, and airline travel-planning systems.

Eg. Airline travel-planning web site

- **States** → locations and current time
- **Initial state** → home (source) airport
- **Goal state** → destination city (airport)
- **Actions** → Take any flight from the current location, in any seat class, leaving after the current time, leaving enough time for within-airport transfer if needed.
- **Transition model** → The state resulting from taking a flight will have the flight's destination as the new location and the flight's arrival time as the new time.
- **Action cost** → A combination of monetary cost, waiting time, flight time, customs and immigration procedures, seat quality, time of day, type of airplane, frequent-flyer reward points, and so on.
- Contingency plans — for delayed flight and missed connection



EXAMPLE PROBLEMS

2. Real-world problems

- Touring problems → describe a set of locations that must be visited → eg. traveling salesperson problem (TSP) → a touring problem in which every city on a map must be visited. The aim is to find a tour with cost $< C$
- VLSI layout problem → positioning millions of components and connections on a chip to minimize area, minimize circuit delays, minimize stray capacitances, and maximize manufacturing yield → has cell layout and channel routing.
- Robot navigation → is a generalization of the route-finding problem → Rather than following distinct paths (such as the roads in Romania), a robot can roam around, in effect making its own paths.
- Automatic assembly sequencing → Minimizing the amount of manual human labor on the assembly line can produce significant savings in time and cost → eg. protein design, in which the goal is to find a sequence of amino acids that will fold into a three-dimensional protein with the right properties to cure some disease.

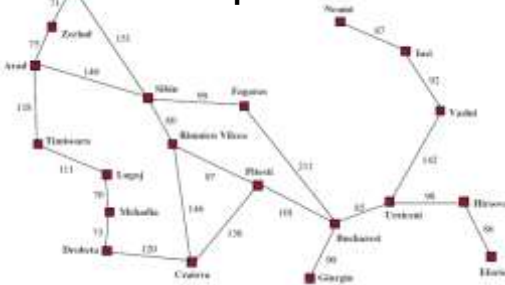
SEARCH ALGORITHMS

- A **search algorithm** → takes a search problem as input and returns a solution, or an indication of failure.
- A **search tree** over the state space graph is used, forming various paths from the initial state, trying to find a path that reaches a goal state.
- Each **node** in the search tree corresponds to a state in the state space and the edges in the search tree correspond to actions.
- The **root** of the tree corresponds to the initial state of the problem.
- The **state space** → describes the (possibly infinite) set of states in the world, and the actions that allow transitions from one state to another.
- The **search tree** describes paths between these states, reaching towards the goal → may have multiple paths to (and thus multiple nodes for) any given state, but each node in the tree has a unique path back to the root (as in all trees).

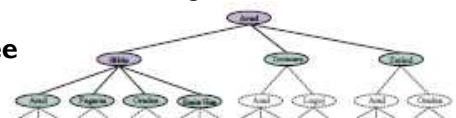
SEARCH ALGORITHMS

- To construct a search tree, start with a root node, apply RESULT function, check where the action leads to and generate a new node (called a child node or successor node).
- Frontier → a set of unexpanded nodes → separates two regions of the state-space graph: an interior region where every state has been expanded, and an exterior region of states that have not yet been reached.

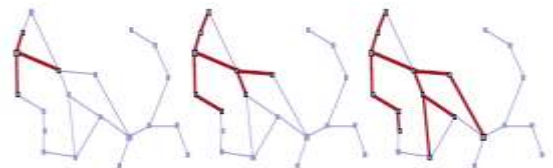
A Map



A search tree



Sequence of search trees generated by a graph search



BEST FIRST SEARCH

- Evaluation function, $f(n)$ is used to find the next possible node.
- On each iteration we choose a node on the frontier with minimum $f(n)$ value, return it if its state is a goal state, and otherwise apply EXPAND to generate child nodes.
- Each child node is
 - Added to the frontier if it has not been reached before, or
 - Re-added if it is now being reached with a path that has a lower path cost than any previous path
- The algorithm returns
 - Either an indication of failure,
 - Or a node that represents a path to a goal.

SEARCH DATA STRUCTURES

- A **node** in the tree is represented by a data structure with four components:
 - node.STATE: the state to which the node corresponds;
 - node.PARENT: the node in the tree that generated this node;
 - node.ACTION: the action that was applied to the parent's state to generate this node;
 - node.PATH-COST: the total cost of the path from the initial state to this node. In mathematical formulas, $g(\text{node})$ is used as a synonym for PATH-COST.
- Following the PARENT pointers back from a node allows us to recover the states and actions along the path to that node.
- Doing this from a goal node gives us the solution.

SEARCH DATA STRUCTURES

- Data structure to store the **frontier** has the following:
 - IS-EMPTY(frontier) returns true only if there are no nodes in the frontier.
 - POP(frontier) removes the top node from the frontier and returns it.
 - TOP(frontier) returns (but does not remove) the top node of the frontier.
 - ADD(node, frontier) inserts node into its proper place in the queue
- Kinds of **queues** used in search algorithms:
 - A priority queue first pops the node with the minimum cost according to some evaluation function, $f \rightarrow$ used in best-first search.
 - A FIFO queue or first-in-first-out queue first pops the node that was added to the queue first $\rightarrow$ used in breadth-first search.
 - A LIFO queue or last-in-first-out queue (or a stack) pops first the most recently added node $\rightarrow$ used in depth-first search.

REDUNDANT PATHS

- Repeated state $\rightarrow$ a node generated again after few searches
- It causes a loopy path / cycle.
- Redundant path $\rightarrow$ when 2 paths exist from source to destination, 1 is ignored because of being a worse way.
- Algorithms must remember the past so as to not to repeat it.
- 3 approaches to solve this issue:
 - Remember all previously reached states $\rightarrow$ must fit in memory
 - No worry about repeating the past $\rightarrow$ search algorithm is a graph search if it checks for redundant; it is tree-like search paths if it does not check.
 - Compromise time and check for cycles.

MEASURING PROBLEM-SOLVING PERFORMANCE

- Evaluate an algorithm's performance in the following ways:
 - **Completeness:** Is the algorithm guaranteed to find a solution when there is one, and to correctly report failure when there is not?
 - **Cost optimality:** Does it find a solution with the lowest path cost of all solutions?
 - **Time complexity:** How long does it take to find a solution? This can be measured in seconds, or more abstractly by the number of states and actions considered.
 - **Space complexity:** How much memory is needed to perform the search?
- A search algorithm must be systematic in the way it explores an infinite state space, making sure it can eventually reach any state that is connected to the initial state.
- For an implicit state space, complexity can be measured in terms of
 - d , the depth or number of actions in an optimal solution
 - m , the maximum number of actions in any path
 - b , the branching factor or number of successors of a node that need to be considered.

UNINFORMED SEARCH STRATEGIES

UNINFORMED SEARCH STRATEGIES

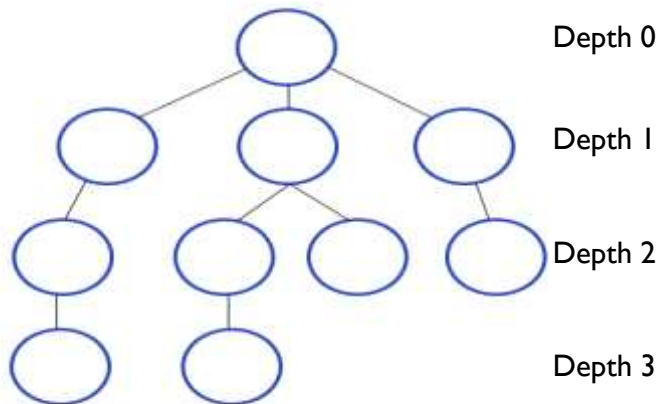
- An uninformed search algorithm is given no clue about how close a state is to the goal(s).
- A class of search algorithms that do not have any additional information about the problem other than what is given in the problem definition.
- Play a fundamental role in AI.
- The main role → to systematically explore the search space to find a solution, without using any domain-specific knowledge or heuristics.
 - 1. Breadth-first search
 - 2. Dijkstra's algorithm or uniform-cost search
 - 3. Depth-first search and the problem of memory
 - 4. Depth-limited and iterative deepening search
 - 5. Bidirectional search

BREADTH FIRST SEARCH (BFS)

- Preferred when all actions have the same cost
- Expand the root node, then its successors, and so on.
- A systematic search strategy
- Evaluation function $f(n)$ used → **the depth of the node** → the number of actions it takes to reach the node
- **Frontier** → a **queue** → used as data structure for processing the nodes in FIFO pattern – old nodes dequeued and expanded first.
- **Reached** → a set of states that decide the goal state
- **Early goal test** → checking whether a node is a solution as soon as it is generated.
- Implemented as a tree-like search
- Breadth-first search always finds a solution with a minimal number of actions, because when it is generating nodes at depth d , it has already generated all the nodes at depth $d-1$, so if one of them were a solution, it would have been found.

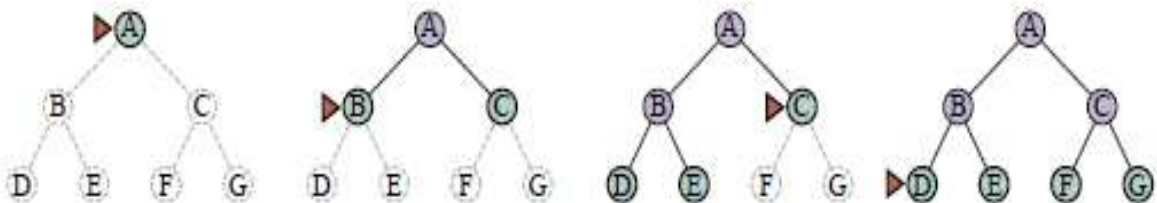
BREADTH FIRST SEARCH

When BFS algorithm is generating nodes at depth 2, it has already generated all the nodes at depth 1



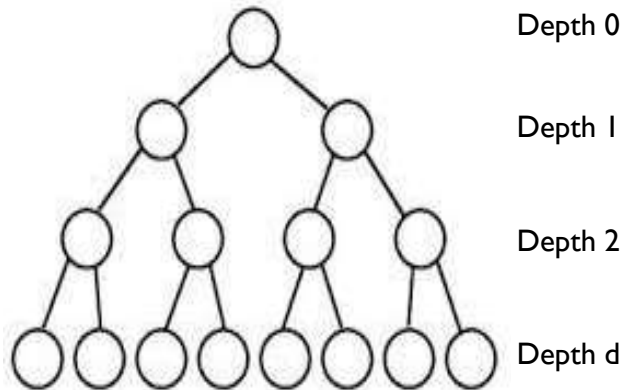
BREADTH FIRST SEARCH

Example:



BREADTH FIRST SEARCH

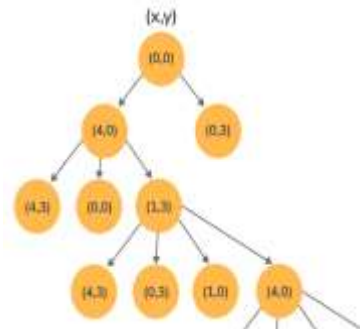
If solution is at depth d ,
 the no. of nodes generated
 $= 1 + b + b^2 + b^3 + \dots + b^d$
 $= O(b^d)$
 ➔ time and space complexity



BREADTH FIRST SEARCH

```

function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
    node ← NODE(problem.INITIAL)
    if problem.IS-GOAL(node.STATE) then return node
    frontier ← a FIFO queue, with node as an element
    reached ← {problem.INITIAL}
    while not IS-EMPTY(frontier) do
        node ← POP(frontier)
        for each child in EXPAND(problem, node) do
            s ← child.STATE
            if problem.IS-GOAL(s) then return child
            if s is not in reached then
                add s to reached
                add child to frontier
    return failure
    
```

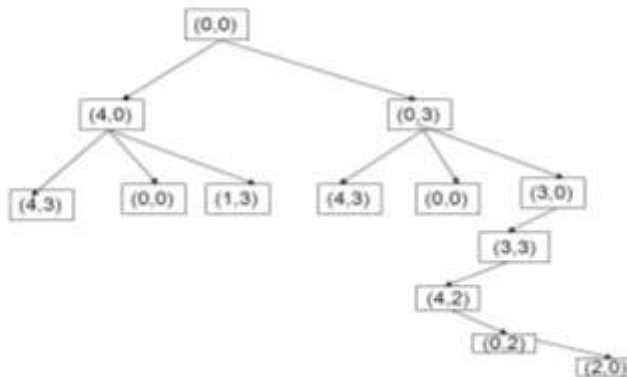


```

function UNIFORM-COST-SEARCH(problem) returns a solution node, or failure
    return BEST-FIRST-SEARCH(problem, PATH-COST)
    
```

BREADTH FIRST SEARCH

BFS Tree for Water Jug problem



BREADTH FIRST SEARCH

For the given tree, let 5 has the goal state:

Procedure for BFS:

If 1 has the goal state (early goal test), return 1 as solution.

Otherwise put 1 in the FIFO queue (frontier)

reached = { 1's state }

while frontier is not empty,

pop 1 and expand 1 (process its child nodes 2, 3 one by one)

if 2's state is goal state, return 2

else reached = { 1's state, 2's state }, put 2 in frontier

if 3's state is goal state, return 3

else reached = { 1's state, 2's state, 3's state }, put 3 in frontier

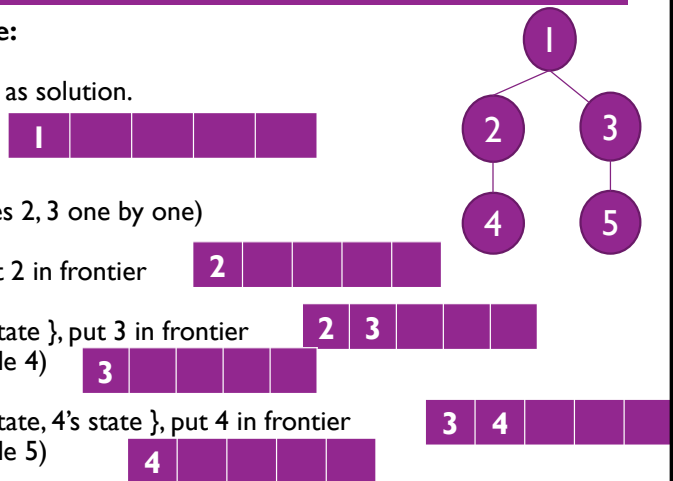
pop 2 and expand 2 (process its child node 4)

if 4's state is goal state, return 4

else reached = { 1's state, 2's state, 3's state, 4's state }, put 4 in frontier

pop 3 and expand 3 (process its child node 5)

if 5's state is goal state, return 5



BREADTH FIRST SEARCH

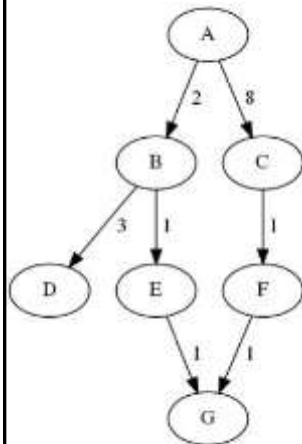
Drawbacks:

- The memory requirements - a bigger problem for breadth-first search than the execution time.
- So exponential-complexity search problems cannot be solved by uninformed search for any but the smallest instances

DIJKSTRA'S ALGORITHM OR UNIFORM-COST SEARCH (UCS)

- Preferred when all **actions** have different costs
- **Evaluation function** $f(n)$ → finds the cost of the path from the root to the current node.
- Called as uniform-cost search algorithm – because it spreads out in waves of uniform path-cost. (where BFS spreads out in waves of uniform depth, depth-by-depth)
- Implemented as a graph search (rather than as a tree-like search)
- UCS expands (child nodes are identified) the node with the lowest path cost.
- **Late goal test** → checks whether a node is a solution only when it is expanded (and not when generated)
- Uses a priority queue as data structure.
- If all actions have the same cost, UCS algorithm works similar to BFS.

DIJKSTRA'S ALGORITHM OR UNIFORM-COST SEARCH



Uniform Cost Search uses the total weight of the edges from the root to the node.

It works as follows:

1. Visit A:0, add B:2 and C:8 to the priority queue: [B:2, C:8].

2. Visit B:2, add D:5 and E:3 to the priority queue: [E:3, D:5, C:8].

3. Visit E:3, add G:4 to the priority queue: [G:4, D:5, C:8].

4. Visit G:4.

5. Visit D:5.

6. Visit C:8, add F:9 to the priority queue: [F:9].

7. Visit F:9, ignoring child G:10 because it has already been visited in step 4.

Shortest path generated by UCS → A B E G D C F

Shortest path generated by BFS → A B C D E F G

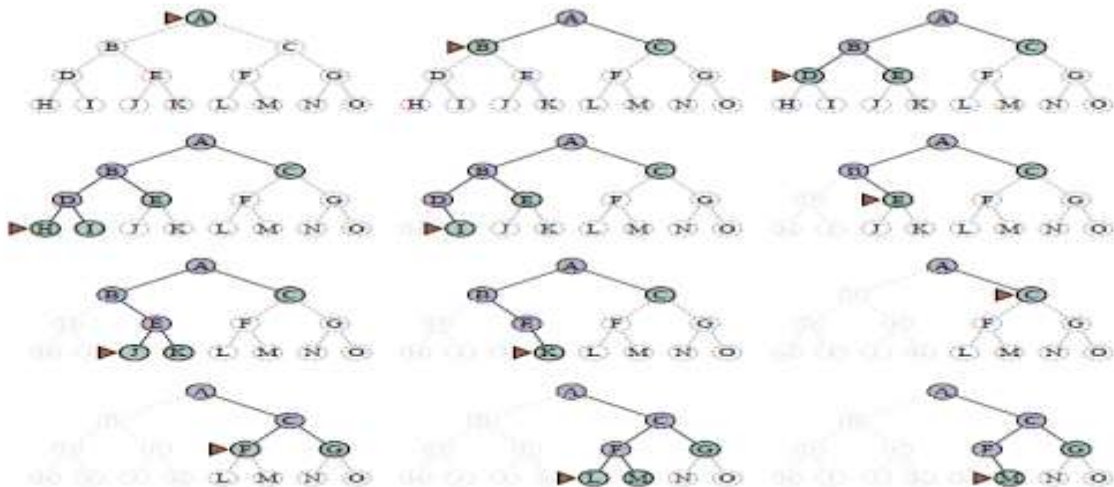
DIJKSTRA'S ALGORITHM OR UNIFORM-COST SEARCH

- The complexity of uniform-cost search is characterized in terms of C^* → the cost of the optimal solution
- ϵ → a lower bound on the cost of each action, with $\epsilon > 0$. Then the worst-case time and space complexity = $O(b^{1+\lceil C^*/\epsilon \rceil}) b^d$
- Uniform-cost search is complete and is cost-optimal, because the first solution it finds will have a cost that is at least as low as the cost of any other node in the frontier.
- Uniform-cost search considers all paths systematically in order of increasing cost, never getting caught going down a single infinite path.

DEPTH FIRST SEARCH AND THE PROBLEM OF MEMORY (DFS)

- DFS always expands the deepest node in the frontier first.
- Evaluation function $f(n) \rightarrow$ the negative of the depth.
- Implemented as a tree-like search and keeps a table of reached states –
- Basic strategy:
 - Search proceeds immediately to the deepest level of the search tree, where the Nodes have no successors.
 - The search then “backs up” to the next deepest node that still has unexpanded successors.
- DFS is not cost-optimal; it returns the first solution it finds, even if it is not cheapest.
- Example tree..

DEPTH FIRST SEARCH AND THE PROBLEM OF MEMORY (DFS)



DEPTH FIRST SEARCH AND THE PROBLEM OF MEMORY (DFS)

Finite state spaces

- For acyclic state spaces → DFS may end up expanding the same state many times via different paths, but will systematically explore the entire space → DFS is efficient and complete.
- In cyclic state spaces → DFS can get stuck in an infinite loop; therefore some implementations of depth-first search check each new node for cycles → DFS is incomplete

For infinite state spaces

- DFS → not systematic: it can get stuck going down an infinite path, even if there are no cycles → DFS is incomplete.

DEPTH FIRST SEARCH AND THE PROBLEM OF MEMORY (DFS)

DFS preferred over BFS – why?

- DFS needs less memory for tree-like search.
- No reached table
- Small frontier
- Space complexity → $O(bm)$ b → branching factor, m → maximum depth of the tree

DEPTH FIRST SEARCH AND THE PROBLEM OF MEMORY (DFS)

- A variation of DFS → **backtracking search** → only one successor is generated at a time rather than all successors; each partially expanded node remembers which successor to generate next
- Undo each action when we backtrack → modifying the current state description directly – so less memory used.
- Backtracking is critical to the success of many problems with large state descriptions, such as robotic assembly.
- Space complexity → $O(m)$

DEPTH LIMITED AND ITERATIVE DEEPENING SEARCH

- Used to keep depth-first search from travelling down an infinite path.
- **Depth limit (or) diameter of the state-space graph** ℓ → In DFS treat nodes at level ℓ as if they had no successors .
- Time complexity = $O(b^\ell)$, space complexity = $O(b\ell)$
- Poor choice for ℓ → the algorithm will fail to reach the solution, making it incomplete.
- Good choice → made based on knowledge of the problem → **diameter of the state-space graph**, gives us a better depth limit, which leads to a more efficient depth-limited search
- Iterative deepening search → solves the problem of picking a good value for ℓ by trying all values: first 0, then 1, then 2, and so on - until either a solution is found, or the depth limited search returns the failure value rather than the cutoff value. (to mean there might be a solution at a deeper depth than ℓ).
- Optimal for problems where all actions have the same cost.
- Time complexity = $O(b^d)$ (when there is solution), time complexity = $O(b^m)$ (when there is no solution)

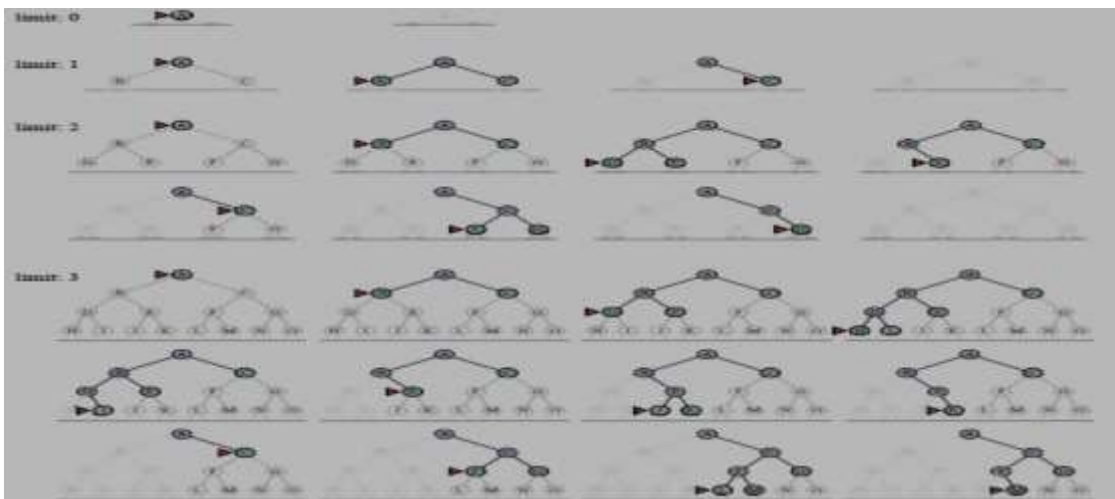
DEPTH LIMITED AND ITERATIVE DEEPENING SEARCH

function ITERATIVE-DEEPENING-SEARCH(*problem*) **returns** a solution node or *failure*
for *depth* = 0 **to** ∞ **do**
 result $\leftarrow$ DEPTH-LIMITED-SEARCH(*problem*, *depth*)
 if *result* $\neq$ *cutoff* **then return** *result*

function DEPTH-LIMITED-SEARCH(*problem*, ℓ) **returns** a node or *failure* or *cutoff*
frontier $\leftarrow$ a LIFO queue (stack) with NODE(*problem*.INITIAL) as an element
result $\leftarrow$ *failure*
while **not** IS-EMPTY(*frontier*) **do**
 node $\leftarrow$ POP(*frontier*)
 if *problem*.IS-GOAL(*node*.STATE) **then return** *node*
 if DEPTH(*node*) > ℓ **then**
 result $\leftarrow$ *cutoff*
 else if **not** IS-CYCLE(*node*) **do**
 for each *child* **in** EXPAND(*problem*, *node*) **do**
 add *child* to *frontier*
return *result*

DEPTH LIMITED AND ITERATIVE DEEPENING SEARCH

Eg.



DEPTH LIMITED AND ITERATIVE DEEPENING SEARCH

- Each iteration of iterative deepening search generates a new level like BFS
- But BFS stores all nodes in memory, while iterative deepening generates new levels by repeating the previous levels → saves memory at the cost of more time.
- States near the top of the search tree are re-generated multiple times
- Total number of nodes generated in the worst case,
$$N(IDS) = (d)b^1 + (d-1)b^2 + (d-2)b^3 \dots + b^d$$
- If $b=10$ and $d=5$, $N(IDS) = 123450$ and $N(BFS) = 111110$
- Iterative deepening → the preferred uninformed search method when the search state space is larger than can fit in memory and the depth of the solution is not known.
- Hybrid approach → BFS (for all memory) + IDS (for the frontier)

BIDIRECTIONAL SEARCH

- Simultaneously searches forward from the initial state and backwards from the goal state(s), hoping that the two searches will meet.
- Keep track of two frontiers and two tables of reached states.
- Solution available when the two frontiers (from forward and backward directions) collide → a's successor is b and b's successor is a.
- Bidirectional best first search → node to be expanded next is always one with a minimum value of the evaluation function, across either frontier.
- If the cost of the optimal path is C^* , then no node with cost $> C^*/2$ will be expanded.
- Keep track of the best solution found so far, and update that several times, if needed.

a

b

COMPARING UNINFORMED SEARCH STRATEGIES

- A comparison for tree-like search versions which don't check for repeated states.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

- For graph searches (check for repeated states) → DFS is complete for finite state spaces, and the space and time complexities are bounded by the size of the state space.

COMPARING UNINFORMED SEARCH STRATEGIES

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

1. Complete → if b is finite, the state space either has a solution or is finite
2. Complete → if all action costs $\geq \epsilon > 0$; (ϵ = lower bound on the cost action)
3. Cost-optimal if action costs are all identical;
4. Space → if both directions are breadth-first or uniform-cost.

INFORMED (HEURISTIC) SEARCH STRATEGIES

49

INFORMED SEARCH STRATEGIES

- An informed search strategy → one that uses domain-specific hints about the location of goals → can find solutions more efficiently than an uninformed strategy is given no clue about how close a state is to the goal(s).
- The hints come in the form of a heuristic function, denoted $h(n)$:
 $h(n)$ = estimated cost of the cheapest path from the state at node n to a goal state.
- Eg. In route-finding problems, estimation of the distance from the current state to a goal by computing the straight-line distance on the map between the two points
 1. Greedy best-first search
 2. A* search
 3. Search contours
 4. Satisficing search: Inadmissible heuristics and weighted A*
 5. Bidirectional heuristic search

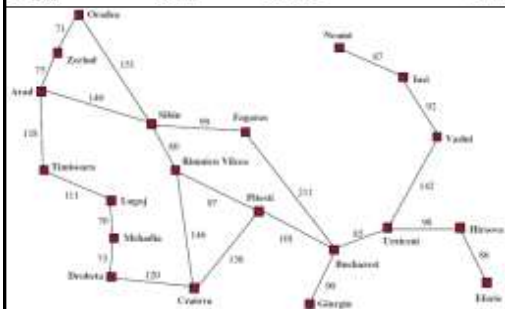
GREEDY BEST - FIRST SEARCH

- A form of best-first search that uses the evaluation function $f(n) = h(n)$
- It expands first the node with the lowest $h(n)$ value $\rightarrow$ the node that appears to be closest to the goal (likely to lead to a solution quickly)
- “Greedy” $\rightarrow$ on each iteration it tries to get as close to a goal as it can, but greediness can lead to worse results than being careful.
- Eg. Route-finding problems in Tamil Nadu $\rightarrow$ using straightline distance heuristic h_{SLD}
- If goal=Chennai, $h_{SLD}(\text{Ooty}) = 555$
- h_{SLD} cannot be computed from the problem description $\rightarrow$ needs certain amount of world knowledge.
- Eg. Route-finding problems in Romania $\rightarrow$ from Arad to Bucharest

CITY	DISTANCE (KM)
Trichy	330
Chennai	0
Madurai	462
Ooty	555
Coimbatore	505

GREEDY BEST - FIRST SEARCH

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



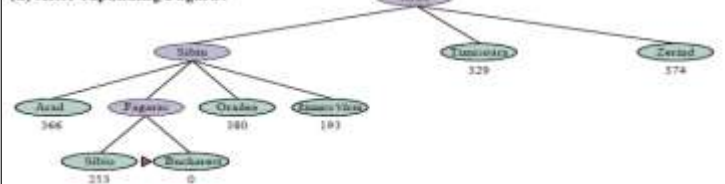
(a) The initial state



(b) After expanding Arad



(c) After expanding Sibiu



GREEDY BEST - FIRST SEARCH

Progress of greedy best-first search algorithm to find route in Romania map → from Arad to Bucharest

- Initial node (root) → Arad.
- The first node to be expanded from Arad → Sibiu because the heuristic says it is closer to Bucharest than is either Zerind or Timisoara.
- The next node to be expanded from Sibiu → Fagaras because it is now closest according to the heuristic.
- Fagaras in turn generates Bucharest → goal reached.
- For this particular problem, greedy best-first search using h_{SLD} finds a solution without ever expanding a node that is not on the solution path.
- The solution found does not have optimal cost, however: the path via Sibiu and Fagaras to Bucharest is 32 miles longer than the path through Rimnicu Vilcea and Pitesti.

GREEDY BEST - FIRST SEARCH

Analysis

- Greedy best-first graph search is **complete** in finite state spaces, but not in infinite ones.
- The worst-case time and space complexity is $O(|V|)$.
- With a good heuristic function $h(n)$, the complexity can be reduced substantially, on certain problems reaching $O(bm)$.

A* SEARCH

- A form of best-first search that uses the evaluation function

$$f(n) = g(n) + h(n)$$

where $g(n) \rightarrow$ the path cost from the initial state to node n ;

$h(n) \rightarrow$ the estimated cost of the shortest path from n to a goal state

$f(n) \rightarrow$ the estimated cost of the best path that continues from n to a goal.

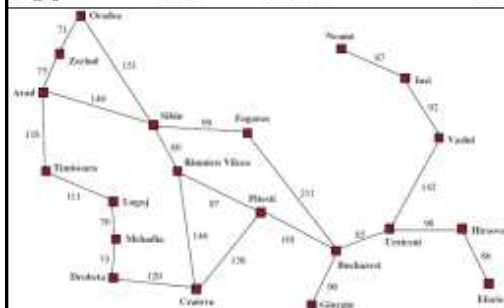
- Eg. Route-finding problems in Romania $\rightarrow$ from Arad to Bucharest

Progress of greedy best-first search algorithm to find route in Romania map $\rightarrow$ from Arad to Bucharest

- At Step-b $\rightarrow$ Arad is expanded as Sibiu, Zerind or Timisoara $\rightarrow$ for each $f(n)$ is computed $\rightarrow f(\text{Sibiu}) = g(\text{Sibiu}) + h(\text{Sibiu}) = 140 + 253 = 393$; Similarly, $f(\text{Zerind}) = 449$ and $f(\text{Timisoara}) = 447$.
- At Step-c $\rightarrow$ The lowest cost node Sibiu is expanded as Arad, Fagarus, Oradea and Rimnicu Vilcea
- At Step-d $\rightarrow$ the lowest cost node Rimnicu Vilcea is expanded.
- At Step-e $\rightarrow$ Fagarus is expanded and Bucharest first appears on the frontier, but it is not selected for expansion (and thus not detected as a solution) because at $f = 450$ it is not the lowest-cost node on the frontier—that would be Pitesti, at $f = 417$.
- At Step-f $\rightarrow$ a different path to Bucharest is now the lowest-cost node, at $f = 418$, so it is selected and detected as the optimal solution.

A* SEARCH

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



(a) The initial state



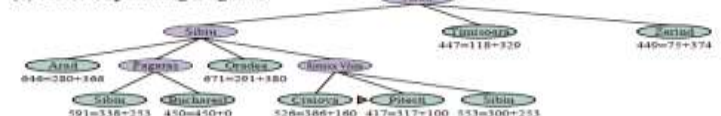
(c) After expanding Sibiu



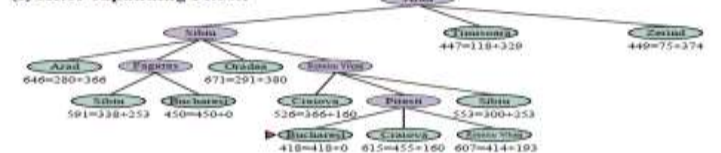
(d) After expanding Rimnicu Vilcea



(e) After expanding Fagaras



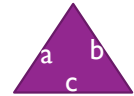
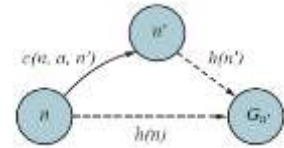
(f) After expanding Pitesti



A* SEARCH

Analysis

- A* search is **complete**.
- A is cost-optimal → depends on the heuristic.
- **Admissibility** → a key property of heuristic → an admissible heuristic is one that never overestimates the cost to reach a goal → therefore optimistic → A* returns only cost-optimal paths.
- **Consistency** → Another property of heuristic → $h(n)$ is consistent if, for every node n and every successor n' of n generated by an action a , we have: $h(n) \leq c(n, a, n') + h(n')$ → a form of triangle inequality.
(Triangle inequality → in a triangle $a \leq b + c$)



SATISFICING SEARCH: INADMISSIBLE HEURISTICS AND WEIGHTED A*

- A* search → has many good qualities → but it expands a lot of nodes.
- Fewer nodes can be explored (taking less time and space) if we are willing to accept satisficing solutions.
- **Satisficing solutions** → solutions that are suboptimal, but are “good enough”.
- A* search is admissible → means it never overestimates the cost to reach a goal → uses SLD.
- If A* is used as an inadmissible heuristic → one that may overestimate → then the optimal heuristic solution may be missed → but the heuristic can be more accurate, and the number of nodes expanded may be reduced.
- Using **detour index** → a multiplier applied to the SLD to account for the typical curvature of roads.

SATISFICING SEARCH: INADMISSIBLE HEURISTICS AND WEIGHTED A*

- Weighted A* search → uses this concept of **detour index**.
- The evaluation function for weighted A* search is $f(n) = g(n) + W h(n)$ for some $W > 1$.
- It expands fewer states – but finds a costlier solution → if the optimal solution costs C^* , a weighted A* search will find a solution that costs between C^* and $W C^*$.
- C^* → the action cost / distance from initial state to goal state.
- So based on evaluation functions used, weighted A* is a generalization of the other algorithms:
 - A* search: $f(n) = g(n) + h(n) \rightarrow (W = 1)$
 - Uniform-cost search: $f(n) = g(n) \rightarrow (W = 0)$
 - Greedy best-first search: $f(n) = h(n) \rightarrow (W = \infty)$
 - Weighted A* search: $f(n) = g(n) + W h(n) \rightarrow (1 < W < \infty)$

SATISFICING SEARCH: INADMISSIBLE HEURISTICS AND WEIGHTED A*

Variations of suboptimal search algorithms:

- Bounded suboptimal search → guarantees a search solution that is within a constant factor W of the optimal cost. → eg. Weighted A*.
- Bounded-cost search → guarantees a solution whose cost is less than some constant C .
- Unbounded-cost search → guarantees a faster search and a solution of any cost is accepted → eg. speedy search

MEMORY BOUNDED SEARCH

- An issue with A* search → uses more memory.
- Some implementation tricks can save space.
- **Best-first search:** Some states are stored in 2 places → as node in the frontier and as an entry in the table of reached states.
- If frontier is smaller than reached, then duplication is not an issue.
- Sometimes store a state in any one place, but the alg. consumes more time.
- Some algorithms eliminate reached states table and use only frontier.
- **Reference counts** → used by some alg. → the number of times a state has been reached, and remove it from the reached table when there are no more ways to reach the state.

MEMORY BOUNDED SEARCH

Algorithms designed to conserve memory usage

- Beam search
- Iterative-deepening A* search (IDA*)
- Recursive best-first search (RBFS)
- Memory-bounded (MA*)
- Simplified MA* (SMA*).

Beam Search

- Limits the size of the frontier
- Keeps only the k nodes with the best $f(n)$ scores, discarding any other expanded nodes.
- Explores only a focused portion that contains the k best candidates.

MEMORY BOUNDED SEARCH

Iterative-deepening A* search (IDA*)

- Keeps all reached states in memory → at a cost of visiting some states multiple times.
- The cutoff = the f-cost ($g+h$) → at each iteration, the cutoff value is the smallest $f(n)$ of any node that exceeded the cutoff on the previous iteration.

Recursive best-first search (RBFS)

- Resembles a recursive depth first search.
- But uses the f limit variable to keep track of the f-value of the best alternative path available from any ancestor of the current node.
- If the current node exceeds this limit, the recursion unwinds back to the alternative path → but remembers the f -value of the best leaf in the forgotten subtree and can therefore decide whether it's worth reexpanding the subtree at some later time.

MEMORY BOUNDED SEARCH

Simplified MA* (SMA*).

- Proceeds just like A* → expands the best leaf until memory is full → an old node must be dropped (the worst leaf node is dropped) to add a new node to the search tree.
- RBFS backs up the value of the forgotten node to its parent.
- The ancestor of a forgotten subtree knows the quality of the best path in that subtree.
- SMA* regenerates the subtree only when all other paths have been shown to look worse than the path it has forgotten.